

Change Report — Agent Work

What was agreed, what was built, and what it was checked against

12 September 2026
5 commits · 5 of 5 items
260 tests passing

All five agreed items are complete. The work is on `github.com/Atharvsk001/VibrationMonitoring`, branch `main`.

Two of the five were the substantive ones: making the data path adapt to the database instead of carrying a hand-written list of fields, and separating the ISO limits lookup into an agent of its own. Along the way five defects were found and fixed — three of them in work done earlier in this same effort, found only because each claim was tested rather than reviewed.

1. The five items

#	AGREED	DELIVERED	STATUS
1	Make the SQL agent dynamic — read the database structure, no hardcoded columns, so it survives a change to the database	Export derives its columns from the data. 27 → 97 columns. Backend response models opened too; 100% column coverage on all three tables	DONE
2	Take the ISO tool out of the knowledge-base agent; keep it as a totally separate agent	<code>iso_agent/</code> , verified independent. No language model participates at all	DONE
3	Verify the RPM calculation against the backend — how it is made, how it is carried out	Three independent checks against the raw file. One real defect found	DONE
4	A terminal runner, so running one agent does not alter another's data	<code>run_agent.py</code> . Isolation is <i>checked</i> , proven by forcing a violation	DONE
5	Two documents — one plain language, one technical	Both in the repository, updated for the four-agent layout	DONE

2. Item 1 — the data path is now schema-driven

What was wrong

Three separate layers each carried a hand-written list of fields, and each silently discarded anything not on its own list. No error, no warning.

This had already cost us the shaft speed. The platform computes it and stores it in a field called `metadata`. That field was on none of the three lists, so it was thrown away in transit and the machine's speed appeared to be unknown — while sitting in the database the whole time.

What changed

LAYER	BEFORE	AFTER
Export (<code>sql_agent</code>)	27 names, <code>extrasaction="ignore"</code>	Columns derived from the data received
API (<code>response_model</code>)	Whitelist of declared fields	<code>RowPassthrough</code> carries every mapped column
Database model	Declared	Still declared, deliberately — see below

Measured result

TABLE	COLUMNS IN THE DATABASE	REACHING THE EXPORT BEFORE	AFTER
<code>equipment_masters</code>	47	44	47
<code>sensor_data_uploads</code>	21	18	21
<code>measurement_channel_features</code>	10	7	10

Export overall: **27 → 97 columns**. Sixty-one fields had been discarded, including the ones that matter most — nameplate speed and power, both bearing part numbers, foundation type and sensor sensitivity. All of those are now available to the report agent without asking the operator for them.

Why the database model was left declared

The obvious next step — have the model mirror the database automatically — was **deliberately not taken**, and this is the one decision in this report worth a second opinion.

The team's migration tool generates migrations by comparing the declared models against the database. If the models simply mirrored the database, that comparison would mean nothing and every migration would have to be written by hand. That is a cost for the whole team, to fix a case that should not arise.

Instead the models remain the source of truth, and a startup check reports any database column they do not declare. **Adding a field used to need edits in three places; it now needs one line in one file**, and if anyone skips that line the backend says so at startup rather than losing the data quietly.

Tested by adding a real column to the database, confirming it did not appear, declaring it on the model, and confirming it then flowed all the way to the CSV with nothing else touched. Both test columns were removed afterwards.

3. Item 2 — the ISO limits agent is now separate

Why it had to be code, not a model

The ISO lookup was threaded through the knowledge-base agent in five places. It was there in code, rather than left to the model, because the model's behaviour had been measured:

- Asked for a 55 kW pump's Zone B/C boundary, it read Table A.2 out of the ingested document and never called the tool. Pumps are Groups 3 and 4 — **that document contains neither, at any power**, so every figure it could find was the wrong row.
- Handed the correct Group 3 (2.3 / 4.5 / 7.1) and Group 4 (1.4 / 2.8 / 4.5) tables side by side, it reported **2.8 for both**.
- Over five runs of one scoped-limit question it chose the right row **once**.

The separated agent keeps that guarantee by going further: no language model participates at all.

The question is read by pattern matching, the limits come from the unit-tested tables, and the answer is assembled by code. It also grades a reading against the table, and places a value exactly on a boundary in the lower zone, per the standard's own convention.

Where the machine group is ambiguous — a pump with no driver arrangement stated — it returns every applicable table and names what the question left out, rather than choosing.

Verified independent in a fresh interpreter: it pulls in no knowledge-base code, no search index, and needs no API key. The knowledge-base agent no longer exposes the tool at all, and points limit questions at the new agent rather than answering them.

The trade is deliberate and worth stating: this agent understands far less of a question than a model would. Phrased unusually, it will not extract the details. That was the price of never getting the number wrong.

4. Item 3 — the RPM calculation, verified

Not reviewed by reading the code — recomputed independently from the raw capture file, three different ways.

CHECK	METHOD	RESULT
A	Reimplement the backend's own algorithm from scratch	Reproduces the stored values exactly. No transport or storage fault
B	Autocorrelation — a wholly different method	Disagreed on 4 of 8 channels
C	Read the spectra directly to settle which method was wrong	The backend was right on 3 of those 4. My own check was the one fooled

Check C is the reason this was worth doing properly. Stopping at check B would have reported three defects that do not exist. Autocorrelation has a weakness mirroring the FFT's — it locks onto sub-harmonics exactly as peak-picking locks onto harmonics.

Verdict: the algorithm and its stored output are sound. Its one real weakness is harmonic lock — on one channel the 2X peak measures three times the fundamental, so "tallest peak" legitimately selects the wrong component — and the cross-channel vote already contains it.

One real defect found

SOURCE	SAMPLING RATE	SHAFT SPEED
The capture file's own timestamps	25,641 Hz	1502.4 rpm
Recorded in the database	25,600 Hz	1500.0 rpm

A **0.16% discrepancy**. Immaterial at this size, but the stored rate is a configured value that is never reconciled against the file, and the entire frequency axis scales linearly with it. It is the mechanism by which a larger error could enter unnoticed.

The nameplate speed is 1480 rpm and the calculation gives 1500, which had been reported as "1.4% *apart* — *strong independent confirmation*". **That overstated it.**

The method reports speed in steps of 187.5 rpm, so 1480 and 1500 fall in the same step and it could not have printed 1480 even if that were exact. What the agreement proves is that the *correct step* was selected — not double it, not a third of it, not noise. That is the check worth having and it passes. It is not a claim of 1.4% accuracy.

5. Item 4 — the terminal runner

```
python run_agent.py list
python run_agent.py iso      "zone B/C for a 55 kW pump, separate driver, rigid"
python run_agent.py sql      data "cooling water pump"
python run_agent.py report build "cooling water pump" --separate-driver
python run_agent.py kb --with-books ask "what causes oil whirl"
```

- **Loads only the agent asked for.** Imports are deferred until the choice is made, so the SQL agent pulls in no search index and no embedding model. It answers in 0.6 s; the knowledge-base agent, which does load all of that, takes 67 s.
- **The books are opt-in.** The knowledge-base agent is the only one that loads the index and calls a paid API, so it refuses to start without an explicit flag.
- **One model** for every agent that uses one, so a run cannot silently cost more than the last.
- **Isolation is checked, not claimed.** Each agent declares the paths it may write; every other agent's paths are fingerprinted before the run and compared after.

The check was proven by forcing a real violation — pointing the report agent's output into the SQL agent's export directory:

```
ISOLATION : FAILED - this run modified sql:data/exports
```

A check that cannot fail proves nothing, so it was made to fail on purpose before being trusted.

6. Defects found and fixed along the way

Three of these were in work done earlier in this same effort. All were found by running things rather than reading them.

DEFECT	WHAT IT WOULD HAVE CAUSED	FOUND BY
The runner's own documented command did not work	Typing the exact command from the help text was refused, and the error printed back the command just typed. A runner flag was reaching the agent instead of the runner.	Running the documented example
A column silently changed meaning	During the export rewrite, <code>created_at</code> vanished and a column holding the sensor's timestamp was labelled as the capture's. Wrong data under a confident name — precisely the failure this project exists to prevent.	Running against live data

DEFECT	WHAT IT WOULD HAVE CAUSED	FOUND BY
Limit questions were not recognised	The pattern required the word "zone" twice, so " <i>zone B/C boundary</i> " — the usual phrasing — was never matched. Present in the original code, fixed in both copies.	A new test
Three READMEs described replaced code	The SQL agent's README still presented the fixed 27-column list as current design. The knowledge-base README claimed two indexed books when there are eight. The main README did not mention the agents at all.	Sweeping the repository
A verifier that could not fail	An earlier version checked the report <i>after</i> substituting values, at which point every figure is a digit in the text. It would have passed everything forever while appearing to work.	Testing against a known-bad input

The common thread, and the practice worth keeping: every one of these passed a reading and failed a run. A check is only worth having once it has been made to fail on purpose.

7. Where things stand

COMPONENT	STATE	NOTE
Data fetcher (<code>sql_agent</code>)	WORKING	0.6 s, 97 columns, nothing missing
Limits checker (<code>iso_agent</code>)	WORKING	Instant, no API key. Will answer any question mentioning a machine, even when limits were not what was asked — minor, and it only ever returns correct tables
Report writer (<code>report_agent</code>)	WORKING	All 8 steps, 36 traceable figures, passes its own verifier
Librarian (<code>kb_agent</code>)	WORKING	67 s per question and a paid API call — hence opt-in
Runner (<code>run_agent.py</code>)	WORKING	Isolation verified by forced failure

What still blocks a diagnostic report, and none of it is software.

- **No healthy reference recording.** The only one on file is the same recording as the measurements — comparing today against today.
- **One alarm limit shared by every machine on site.** The RMS limit is 0.02; readings run 0.024 to 0.424, so the lowest reading is already over it and every reading returns critical. That describes the setting, not the pump. Fixing it needs a database change on the platform.
- **The nine recordings are one recording, loaded nine times** — identical values throughout. No trend and no prediction is possible from a single snapshot.

8. The single most useful thing anyone could provide

Real recordings of the pump, taken on different days, with one of them confirmed as a known-healthy run.

That one activity resolves two of the three blockers at once: it gives a history to trend against, and it supplies the healthy reference the comparison needs. It would also settle an open question about what the stored measurement

values physically represent, which is currently why no severity grade can be assigned.

The remaining blocker — per-machine alarm limits — is platform development work and is independent of the above.

Cheapest remaining item on our side

Connect the report agent to the equipment records. The nameplate speed, rated power, foundation type and both bearing part numbers now reach the export, so the report can stop asking the operator for them and can begin computing bearing fault frequencies. Small, code only, no plant access required.

Change report · vibration monitoring agent work · 12 September 2026.

Five commits on `main` at github.com/Atharvsk001/VibrationMonitoring. Every figure quoted was read from the running system or produced by the verification scripts described above, at the time of writing. Companion documents in the same repository cover the architecture in plain language and in technical detail.